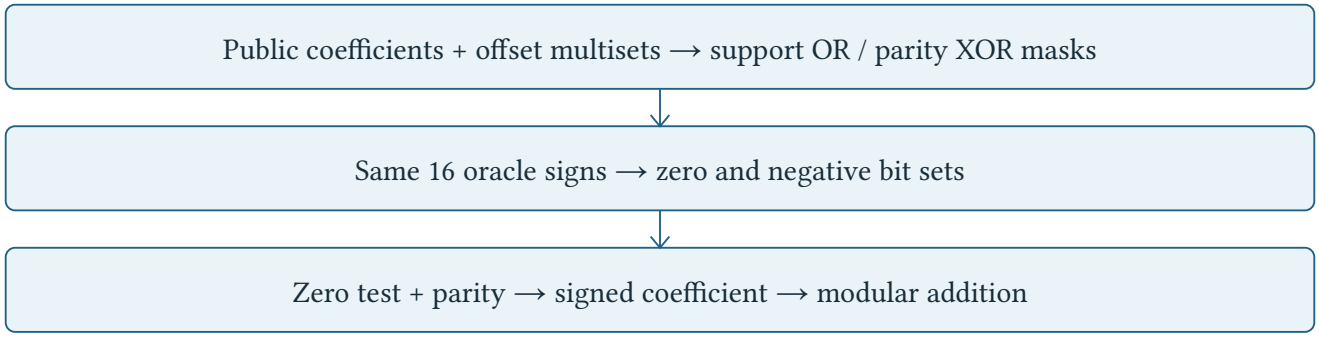
Native signed-character evaluation

2026-11-03 · Exact software checks · No timing comparison

The repaired character polynomial now has a dependency-free Rust evaluator. The compiler stores two masks per monomial: support OR for zero detection, and multiplicity XOR for negative-sign parity. It retains the public oracle boundary and performs no parsing or allocation during evaluation.



Checked: 479 field identities | Unrun: fresh elliptic-source cohort

Identity. If any supported sign is zero, the monomial is zero. Otherwise its sign is negative exactly when the number of negative factors is odd. Separate support and parity masks preserve even exponents at zeros. Empty support represents the constant one. Coefficients and masks are immutable.

Derived arithmetic. The saved program has 477 terms and 1,291 offset occurrences. Direct evaluation performs 1,291 coefficient-by-sign field multiplications. The new evaluator uses zero such multiplications, at most 477 parity computations and modular additions, plus mask and branching work. The 16 oracle calls and 477 terms remain. No elapsed-time saving is measured.

Native checks. At source82fd48f0a683, rustc1.99.1 compiled the standalone unit target; one invocation passed all three methods. The repaired program passes all479 square-set character inputs, agreeing with an independent u128 direct-multiplication evaluator and X. Other checks cover all27 ternary inputs of a polynomial with repeated offsets, canonical-input rejection, width64/high-bit behavior, overflow boundary and constant-only programs.

Limits. Width is at most64; modulus is between3 and half the u64 maximum. The constructor validates representation but does not prove primality or polynomial identity. Expected X remains in the checker. No fresh elliptic source, recovery cohort, workspace test suite or performance panel ran.

Next. Integrate the native sign boundary with the preserved elliptic source contract and unchanged full4,563-case cohort. Native source/runtime publication and authoritative EKS admission remain before that execution. General multiword masks and coefficient synthesis costs are separate work.

Evidence. lib.rs, program.tsv, check-lock.sha256, check-receipt.txt, build.log and test.log. DERIVATION.md states the exact algebra and counted work. This report does not revise historical receipts.

References. NIST DLMF27.9 Rust u64 original preserved decoder and repaired coefficient certificate. This is an elementary sign specialization, with no novelty claim.